



Technical Assignment

Role: Software Architect

Context

You have just joined an early-stage streaming platform as a Senior Software Architect.

Your first assignment is to design and implement a new core module focused on an interactive TV series experience.

This module will serve as a technical reference for future teams, so architectural decisions, code quality, and scalability matter as much as functionality.

Besides the solution scope is limited, please use all this concepts in order to your seniority be correctly assigned.

- SOLID
- CLEAN CODE
- Design patterns
- Modern Software Architecture

Functional Requirements

TV Series Search

- Provide a search interface where users can search for TV series.
- Display results asynchronously (title, year, poster).
- Use the public TVMaze API:

GET <http://api.tvmaze.com/search/shows?q=QUERY>

Series Details

- When a series is selected, display:
 - Poster
 - Summary
 - Genres
 - List of episodes grouped by season

APIs:

GET <http://api.tvmaze.com/shows/{ID}>

GET <http://api.tvmaze.com/shows/{ID}/episodes>

Episode Tracking & Comments

- Users can mark episodes as watched
- Watched state must persist across navigation and reloads
- Users can leave comments on:
 - A series
 - Or a specific episode



Technical Assignment

Role: Software Architect

UI aesthetics are not evaluated, usability is, but if you provide a user friendly and a good UX this will be your diferencial.

AI Feature (Required)

AI-Powered Insights

The system must generate a short AI-based insight for:

- A TV series
- Or a specific episode

The insight should be based on:

- Series or episode summary
- Genres
- (Optional) user comments

Example:

“This episode focuses on ethical dilemmas and character development, appealing to fans of psychological drama.”

Constraints:

- You may use:
 - HuggingFace Inference API (free tier)
 - Any other LLM API, since we can also use to verify your test.
 - The AI integration must:
 - Be isolated behind an interface
 - Handle failures gracefully
 - Have a clear fallback strategy
-

Technical Requirements

- Python
- Architecture: Clean Architecture (or equivalent), SOLID principles
- Networking: Any HTTP client library
- Design: Use appropriate design patterns where applicable
- Testing: Unit tests for business/domain logic
- Docker:
 - Docker Compose v3.3
 - One container for the application
 - One container for the database
 - Application must run on port 7777
- Automation: Everything must run with a single command

Delivery

- Public GitHub repository
- Presentation in PDF
- Include:
 - Dockerfile
 - docker-compose.yml
 - Clear README with setup instructions
- The project will be executed in a Jenkins pipeline inside a VM

Presentation

Once development is complete, you will be asked to present your solution. Prepare a presentation that explains:

- The architectural solution designed
 - The information flow (like a sequence diagram) for the two main features:
 - TV Series Search
 - Series Details
 - How you implemented the AI component
-



Technical Assignment

Role: Software Architect

Evaluation Criteria

CATEGORY	WEIGHT	EVALUATION TIPS
ARCHITECTURE & DESIGN	30%	Separation of concerns, extensibility, trade-offs
CODE QUALITY & MAINTAINABILITY	20%	Readability, SOLID, cohesion
FUNCTIONALITY & STABILITY	30%	Functional correctness, API integration, persistence
AI INTEGRATION QUALITY	10%	Isolation, provider independence, fallback strategy
TESTABILITY	10%	Unit-testable business logic, isolated dependencies

What We Care About

- Clear architectural thinking
- Conscious trade-offs
- Readable, empathetic code
- Reasonable abstractions
- Ownership mindset
- Working Solution

What We Don't Care About

- Pixel-perfect UI
- Completing every possible feature